VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

TECHNICAL DEEP DIVE

Vehicle Make & Model Recognition (VMMR)

A comprehensive study of fine-grained vehicle recognition — covering backbone architectures, embedding generation, metric learning approaches, and the dual-head training paradigm using the Stanford Cars dataset.

1 Introduction to VMMR

What is Fine-Grained Visual Recognition?

Coarse classification distinguishes between fundamentally different visual categories (car vs. truck vs. bus). **Fine-grained visual recognition (FGVR)** operates within a single basic-level category — distinguishing a 2011 BMW 3 Series Sedan from a 2012 BMW 3 Series Sedan, or a Toyota Camry from a Toyota Corolla.

VMMR is an instance of FGVR applied to vehicles. The output is typically a joint label `(Make, Model, Year)`, which maps to one of the N leaf classes in a taxonomy. For Stanford Cars, N = 196.

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

> **Key distinction:** In VMMR the inter-class visual distance can be smaller than intra-class distance. Two instances of the same model photographed from different angles can look more dissimilar than two different models photographed identically. This is what makes VMMR fundamentally harder than standard classification.

Why VMMR is Challenging

Inter-class Similarity

Consecutive model years of the same vehicle often differ only in minor detail — grill shape, badge position, wheel design. Subtle cues invisible to humans at scale must be learned.

Intra-class Variation

The same vehicle model can appear vastly different due to viewpoint, lighting, occlusion, color, after-market modification, and partial capture in surveillance footage.

Long-tail Distribution

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

In real-world datasets, rare models (exotic sports cars, limited editions) have far fewer training examples than common models, causing class imbalance.

Part Localization

Discriminative regions (headlights, grill, badge) are small relative to the full image. Models must implicitly localize these regions without explicit part supervision.

Applications

- **Traffic surveillance & law enforcement:** Identify stolen or suspect vehicles from CCTV
- **Insurance & claims processing:** Verify reported vehicle make/model automatically
- **Autonomous driving:** Understand surrounding vehicle types for behavioral prediction
- **Fleet management:** Automated inventory and tracking by model
- **Parking management:** Permit validation by registered vehicle model
- **Automotive retail:** Visual search ("find cars similar to this photo")

Stanford Cars Dataset

Total Images

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

16,185

Classes (Make/Model/Year)

196

Train / Test Split

8,144 / 8,041

Each image carries a bounding box annotation and a class label. Classes span model years from roughly 2003–2012 across many manufacturers. Due to the roughly 50/50 split, each class has only ~41 training images on average — making transfer learning essential.

Source: Krause et al., "3D Object Representations for Fine-Grained Categorization", ICCV 2013. Available via HuggingFace: `HuggingFaceM4/Stanford-Cars`

2 Backbone Architectures

The backbone is a deep convolutional (or transformer-based) network that maps a raw image tensor to a compact feature vector. For VMMR we almost always initialize from ImageNet-pretrained weights and fine-tune — because 8,144 training images are insufficient to learn good low-level visual features from scratch.

Transfer Learning Rationale

Low-level features (edges, textures, colors) and mid-level features (object parts, surfaces) learned on ImageNet transfer effectively to vehicle images.

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Only the high-level semantic features and the final decision boundary need to be adapted. Fine-tuning typically yields 15–25% absolute accuracy improvement over training from scratch on small datasets like Stanford Cars.

Candidate Backbones

Backbone	Params	Feature dim	Top-1
ResNet-50	25M	2048	76.1%
ResNet-101	44M	2048	77.4%
EfficientNet-B0	5.3M	1280	77.1%
EfficientNet-B3	12M	1536	81.6%
EfficientNetV2-S	21M	1280	83.9%

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Backbone	Params	Feature dim	Top-'
ViT-B/16	86M	768	81.8%
ConvNeXt-Tiny	28M	768	82.1%
DINOv2-ViT-S/14	22M	384	81.1%

> **Recommendation:** Start with **EfficientNet-B0** for rapid prototyping on standard hardware. Scale to **EfficientNetV2-S** or **ConvNeXt-Small** when optimizing for accuracy. For a metric-learning-heavy pipeline, consider **DINOv2** as the backbone since its features are already well-structured for embedding-based retrieval.

▸ **Deep dive: EfficientNet compound scaling**

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

▸ Deep dive: Vision Transformer (ViT) for fine-grained recognition

3 Embedding Generation

An **embedding** is a dense, low-dimensional vector representation of an image such that semantically similar images map to nearby points in the embedding space. The geometry of this space is what metric learning methods directly optimize.

From Backbone to Feature Vector

Most CNN backbones end with a sequence of convolutional blocks producing a spatial feature map of shape `(B, C, H, W)`. Converting this to a fixed-size vector is done via:

Global Average Pooling (GAP)

Average each spatial location over the channel dimension. Produces a `(B, C)` vector. Standard in ResNet, EfficientNet.

$$f_i = (1/HW) \Sigma_{h,w} x_{i,h,w}$$

Retains semantic content; discards spatial position.

Global Max Pooling (GMP)

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Take the maximum activation per channel. More discriminative for detecting the presence of a feature anywhere in the image, but can be noisier.

$$f_i = \max_{h,w} x_{i,h,w}$$

Often combined with GAP in fine-grained tasks.

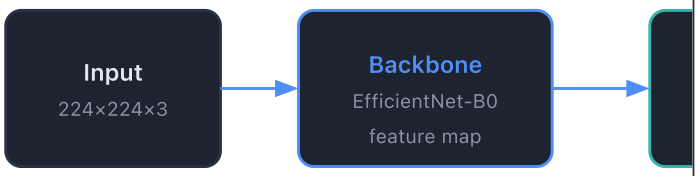

Figure 3.1 — Embedding generation pipeline from raw image to L2-normalized 256-d embedding vector

Projection Head Design

The projection head maps the backbone feature vector (e.g., 1280-d from EfficientNet-B0) to a lower-dimensional embedding space used for metric learning. A common design:

```
class EmbeddingHead(nn.Module):
    def __init__(self, in_features=1280, emb
        super().__init__()
        self.proj = nn.Sequential(
            nn.Linear(in_features, embed_dim
            nn.BatchNorm1d(embed_dim),
        )

    def forward(self, x):
```

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

```
        emb = self.proj(x)
        return F.normalize(emb, p=2, dim=1)
```

Why L2 Normalization Matters

Projecting embeddings onto the unit hypersphere ($\|e\|_2 = 1$) has two critical effects:

- **Bounded distances:** Euclidean distance between unit vectors is in [0, 2], making triplet margins interpretable and training more stable.
- **Cosine similarity as dot product:** For unit vectors, cosine similarity equals the dot product — enabling efficient similarity search with FAISS or ScaNN at inference time.
- **Prevents embedding collapse:** Without normalization, the network can minimize triplet loss by shrinking all embeddings toward zero. L2 normalization eliminates this degenerate solution.

The BN-Neck Trick

> **BN-Neck** (Luo et al., 2019) inserts a BatchNorm layer between the backbone features and the classification/embedding heads. The key insight: the classification head sees the BatchNorm-normalized features (which have better gradient flow for CE loss), while the embedding head optionally operates before normalization. This decouples the two learning objectives partially, stabilizing joint training with combined loss.

Embedding Dimension Choice

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Dim	Use Case	Storage (1M vectors)
64	Highly compressed, retrieval only	256 MB
128	Face recognition (FaceNet standard)	512 MB
256	**Recommended for VMMR**	**1 GB**
512	High accuracy, large-scale retrieval	2 GB

4 Metric Learning Approaches

Metric learning trains the model so that the learned distance function *d(x, y)* reflects semantic similarity: small distance for same-class pairs, large distance for different-class pairs. Below are the five major families, with deep dives on each.

4.1 Contrastive Loss (Siamese Networks)

Operates on *pairs* of images (x_1, x_2) with a binary label $y \in \{0, 1\}$ indicating same or different class.

```
CONTRASTIVE LOSS (HADSELL ET AL., 2006)
L = y · d(e₁, e₂)² + (1 − y) ·
max(0, m − d(e₁, e₂))²
```

For same-class pairs (y=1), the loss minimizes distance. For different-class pairs (y=0), it

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

penalizes if the distance falls below margin m.

Pros

- Simple, well-understood
- Easy to implement
- Natural for verification tasks

Cons

- Requires careful pair construction
- $O(N^2)$ pairs — hard to mine efficiently
- No relative ordering (only absolute threshold)

4.2 Triplet Loss

Operates on *triplets* (anchor a, positive p, negative n): same class for (a,p), different class for (a,n). The objective enforces d(a,p) + margin < d(a,n).

```
TRIPLET LOSS (SCHROFF ET AL., FACENET 2015)
L = Σᵢ max(0, d(aᵢ, pᵢ)² − d(aᵢ, nᵢ)² + α)
```

Mining Strategies

The choice of which triplets to present during training is critical. Easy triplets (already satisfying the margin) provide zero gradient. Hard triplets are

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

the most informative but can cause instability early in training.

Strategy	Positive selection	Negative selectio
Batch Random	Random positive	Random negative
Semi-Hard	Random positive	Neg: d(a,n) > d(a,p), but d(a,n) < d(a,p) + margin
Batch Hard	Hardest positive (max d in batch)	Hardest negative (min d in batch)
Batch All	All positives	All valid negatives

▸ Batch-hard mining: algorithm walkthrough

4.3 ArcFace / CosFace (Angular Margin Losses)

Instead of operating on pairs or triplets, ArcFace/CosFace work within the classification framework but add an angular margin to the softmax logits, directly optimizing the geometry of the embedding space.

```
ARCFACE LOSS (DENG ET AL., 2019)
L = −log [ e^(s · cos(θ_yᵢ + m)) / (e^(s · cos(θ_yᵢ + m)) + Σ_{j≠yᵢ} e^(s · cos(θⱼ))) ]
where θ_yᵢ = angle between embedding and class weight vector, s = scale, m = angular margin
```

```
COSFACE (WANG ET AL., 2018)
```

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

```
L = −log [ e^(s · (cos(θ_yᵢ) − m)) /
(e^(s · (cos(θ_yᵢ) − m)) + Σ_{j≠yᵢ} e^(s
· cos(θⱼ))) ]
```

ArcFace adds the margin *m* to the angle before computing cosine — this creates a stricter angular boundary than CosFace's additive cosine margin, but both achieve state-of-the-art results on face verification and vehicle re-identification benchmarks.

> **Why this is powerful for VMMR:** ArcFace forces the network to separate classes by a minimum angular gap on the hypersphere. The class weight vectors become well-spaced "class prototypes." At inference, classification and embedding-based retrieval both work from the same learned geometry.

4.4 ProxyNCA

ProxyNCA (Movshovitz-Attias et al., 2017) assigns a learnable *proxy* vector to each class. Instead of mining pairs/triplets of images, each image is compared only against these proxies — dramatically reducing the $O(N^3)$ problem of triplet mining to $O(N)$.

```
PROXYNCA LOSS
L(x) = −log [ exp(−d(x, p_{y})) /
Σ_{z≠y} exp(−d(x, p_z)) ]
```

Proxies are trainable parameters and converge to be representative of their class cluster. No mining required; loss is stable and efficient. Particularly well-suited when classes are well-separated (which is not always the case in VMMR).

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

4.5 Supervised Contrastive Loss (SupCon)

SupCon (Khosla et al., NeurIPS 2020) extends self-supervised contrastive learning (SimCLR) to the supervised setting: for each anchor, all same-class images in the batch serve as positives, and all different-class images serve as negatives.

```
SUPCON LOSS
L = Σ_i (−1/|P(i)|) Σ_{p∈P(i)} log [
exp(z_i·z_p/τ) / Σ_{a∈A(i)} exp(z_i·z_a/τ) ]
where P(i) = same-class indices,
A(i) = all other indices, τ =
temperature
```

SupCon is significantly more stable than triplet loss (no fragile mining needed) and often outperforms it. It naturally makes use of all positive pairs in the batch, giving richer gradient signal. Temperature τ (typically 0.07) controls the concentration of the distribution.

Approach Comparison

Approach	Input unit	Mining	Stability
Contrastive	Pairs	HARD ($O(N^2)$)	MEDIUM
Triplet (batch-hard)	Triplets	MEDIUM	MEDIUM
ArcFace	Single image	NONE	HIGH
ProxyNCA	Image + proxies	NONE	HIGH

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Approach	Input unit	Mining	Stability
SupCon	Batch	IMPLICIT	HIGH

5 Dual-Head Architecture

The dual-head architecture trains a single backbone jointly under two complementary objectives: a **classification head** optimized with cross-entropy, and an **embedding head** optimized with triplet loss. The backbone is shared — gradients from both losses propagate through it simultaneously.

Motivation for the Dual-Head Design

- **Cross-entropy alone** produces a classification boundary but may not structure the feature space for retrieval. Similar classes may share a region of the embedding space but aren't explicitly separated.
- **Triplet loss alone** can produce a well-structured embedding space but the training is fragile — requires careful mining and the PKSampler; convergence is slower.
- **Combined:** CE provides stable, fast early convergence (class signal is clear from the beginning). Triplet loss then refines the embedding geometry. The two objectives complement each other.

Architecture Diagram

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

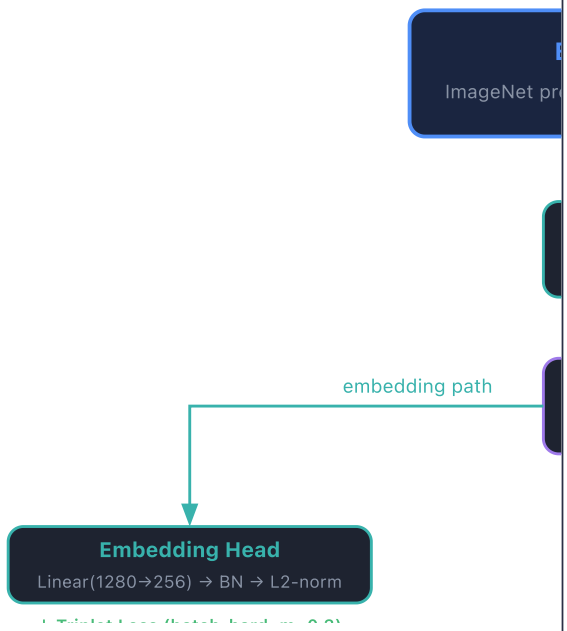

Figure 5.1 — Dual-Head VMMR architecture: shared EfficientNet-B0 backbone with BN-Neck, split into embedding head (triplet loss) and classifier head (CE loss)

Combined Loss

```
TOTAL TRAINING LOSS
L_total = α · L_CE(logits, y, ε=0.1)
+ β · L_triplet(embeddings, y,
m=0.3)

where α, β are loss weights
(default: α=1.0, β=1.0)
```

```
class CombinedLoss(nn.Module):
    def __init__(self, alpha=1.0, beta=1.0,
        super().__init__()
        self.alpha = alpha
        self.beta = beta
        self.ce = nn.CrossEntropyLoss(label_
```

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References


```
        self.margin = margin

    def forward(self, logits, embeddings, la
        l_ce = self.ce(logits, labels)
        l_tri = batch_hard_triplet_loss(embe
        return self.alpha * l_ce + self.beta
```

Alternative Dual-Head Options

Option A — Shared BN-Neck (Recommended)

Both heads branch from a single shared BatchNorm neck after GAP. Heads share the normalized representation.

★ RECOMMENDED — MOST STABLE JOINT TRAINING

Option B — Independent Heads

Both heads attach directly to the raw GAP output with no shared neck. Each head has its own BN/projection.

MODERATE — SLIGHTLY MORE FLEXIBLE, MORE PARAMS

Option C — Triple Head (CE + Triplet + ArcFace)

Adds an ArcFace head alongside CE and triplet. Maximum discriminativeness but sensitive to hyperparameters.

ADVANCED — USE AFTER BASELINE IS WORKING

▸ Model code skeleton (PyTorch)

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

6 PKSampler & Batch Construction

Why Random Sampling Breaks Triplet Loss

Standard random sampling for a 196-class dataset with batch size 32 typically results in each batch containing mostly single-instance classes. With only one image per class per batch, there are *no valid same-class pairs* — the triplet loss has no positives to work with and produces zero useful gradient.

> **Rule of thumb:** For batch-hard triplet mining to work, every batch must contain at least 2 images from the same class. In practice, K ≥ 4 is recommended for stable gradients.

P×K Sampling

The PKSampler (also called "class-balanced batch sampler") constructs each mini-batch by:

1. Randomly sample P classes from the dataset (e.g., P=32)
2. For each selected class, randomly sample K images (e.g., K=4)
3. Resulting batch size = P × K = 128

```
GUARANTEED TRIPLETS PER BATCH
Valid triplets ≥ P · K · (K−1) · (P−1) · K = 32 · 4 · 3 · 31 · 4 = 47,616 triplets
```

Batch-hard mining selects only the hardest P·K = 128 of these — but the pool ensures informative

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

hard triplets are almost always available.

▸ PKSampler implementation

Effect of P and K on Loss Landscape

P (classes/batch)	K (images/class)	Batch size
16	4	64
32	**4**	**128**
32	8	256
64	4	256

7 Training Recipe

Three-Stage Progressive Training

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Progressive fine-tuning avoids destroying pretrained representations by initially keeping the backbone frozen and only training the new heads. As the heads stabilize, deeper backbone layers are progressively unfrozen.

Figure 7.1 — Three-stage progressive training schedule

Hyperparameters

Parameter	Value	Notes
Optimizer	AdamW	weight_decay=1e
Scheduler	CosineAnnealingLR	T_max = total ep
Batch size	P=32 × K=4 = 128	PKSampler
Input size	224 × 224	Use 300×300 fo
Label smoothing (ε)	0.1	nn.CrossEntropy
Triplet margin (m)	0.3	Reasonable for L
Embed dim	256	Balance: accurac
α (CE weight)	1.0	Fixed throughout

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Parameter	Value	Notes
β (triplet weight)	0.5 → 1.0	Ramp up over fir
Dropout	0.3	Applied after BN

Data Augmentation Pipeline

```
train_transform = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.3, c
                           saturation=0.2, h
    transforms.RandomGrayscale(p=0.05),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456,
                         std=[0.229, 0.224,
])

val_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456,
                         std=[0.229, 0.224,
])
```

> **Bounding box crop:** Stanford Cars provides bounding box annotations. Cropping to the bounding box before augmentation can significantly improve accuracy (+3–5% top-1) by removing distracting background. Apply a small padding (e.g., 10%) around the box before cropping.

Loss Weight Scheduling

Starting with a low β for the triplet term avoids the instability common in early triplet training (before

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

embeddings are meaningful, hard triplets are essentially random). Ramping β linearly over the first 10 epochs gives the CE loss time to structure the feature space, after which triplet loss refines the geometry.

```
# Linear β warmup over first `warmup_epochs`
beta_schedule = lambda epoch: min(1.0, 0.5 +
```

8 Evaluation Metrics

Classification Metrics (from Classifier Head)

Top-1 Accuracy

The predicted class (argmax of logits) matches the ground-truth label. Primary metric for VMMR benchmarking.

Stanford Cars SOTA: ~94–95% (with EfficientNetV2 or ViT + strong augmentation)

Top-5 Accuracy

Ground truth is within the top-5 predicted classes. Useful for understanding whether the model is "close" even when wrong. Typically 98–99% for SOTA models.

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

Retrieval Metrics (from Embedding Head)

Recall@K (R@1, R@5)

For a query image, are any of the K nearest neighbors in the embedding space the same class? R@1 ≈ top-1 accuracy for retrieval; R@5 measures the quality of the embedding neighborhood.

mAP@R (Mean Average Precision)

For each query, compute AP over the ranked list of retrieved images of the same class. Average over all queries. More sensitive to the full ranking than R@K.

Embedding Space Visualization

t-SNE or UMAP applied to the 256-d test embeddings gives an intuitive view of the learned space. Good embeddings show tight, well-separated clusters per class. Adjacent clusters should be semantically related (e.g., same model, different year).

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
```

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

```
# Collect embeddings on test set
all_embs, all_labels = collect_embeddings(mo

# Reduce to 2D
tsne = TSNE(n_components=2, perplexity=40, r
embs_2d = tsne.fit_transform(all_embs)

plt.figure(figsize=(14, 10))
scatter = plt.scatter(embs_2d[:, 0], embs_2d
                      c=all_labels, cmap='ta
plt.title("t-SNE of VMMR embeddings (test se
plt.colorbar(scatter)
plt.tight_layout()
plt.savefig("tsne_embeddings.png", dpi=150)
```

Expected Performance Benchmarks

Backbone	Loss	Top-1 Acc	R@1
ResNet-50	CE only	~88%	~84%
EfficientNet-B0	CE only	~91%	~87%
EfficientNet-B0	**CE + Triplet**	**~92%**	**~90%**
EfficientNet-B3	CE + Triplet	~93%	~92%
EfficientNetV2-S	CE + ArcFace	~94%	~93%
DINOv2-ViT-S/14	CE + SupCon	~95%	~94%

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

9 References

1. **Krause, J., Stark, M., Deng, J., & Fei-Fei, L.** (2013). *3D Object Representations for Fine-Grained Categorization.* ICCV Workshop on 3D Representation and Recognition. *[Stanford Cars dataset paper]*
2. **Tan, M., & Le, Q. V.** (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.* ICML 2019. arXiv:1905.11946
3. **Tan, M., & Le, Q. V.** (2021). *EfficientNetV2: Smaller Models and Faster Training.* ICML 2021. arXiv:2104.00298
4. **Schroff, F., Kalenichenko, D., & Philbin, J.** (2015). *FaceNet: A Unified Embedding for Face Recognition and Clustering.* CVPR 2015. arXiv:1503.03832 *[Triplet loss / FaceNet]*
5. **Hermans, A., Beyer, L., & Leibe, B.** (2017). *In Defense of the Triplet Loss for Person Re-Identification.* arXiv:1703.07737 *[Batch-hard mining; PKSampler]*
6. **Deng, J., Guo, J., Xue, N., & Zafeiriou, S.** (2019). *ArcFace: Additive Angular Margin Loss for Deep Face Recognition.* CVPR 2019. arXiv:1801.07698
7. **Wang, H., Wang, Y., Zhou, Z., et al.** (2018). *CosFace: Large Margin Cosine Loss for Deep Face Recognition.* CVPR 2018. arXiv:1801.09414
8. **Movshovitz-Attias, Y., Toshev, A., Leung, T. K., et al.** (2017). *No Fuss Distance Metric Learning using Proxies.* ICCV 2017. arXiv:1703.07464 *[ProxyNCA]*
9. **Khosla, P., Tian, Y., Wang, X., et al.** (2020). *Supervised Contrastive Learning.* NeurIPS 2020. arXiv:2004.11362

VMMR · Deep Dive

1. Introduction
2. Backbone Architectures
3. Embedding Generation
4. Metric Learning
5. Dual-Head Architecture
6. PKSampler & Mining
7. Training Recipe
8. Evaluation
9. References

10. **Luo, H., Gu, Y., Liao, X., Lai, S., & Jiang, W.** (2019). *Bag of Tricks and A Strong Baseline for Deep Person Re-identification*. CVPR Workshops 2019. arXiv:1903.07071 *[BN-Neck trick]*
11. **Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al.** (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021. arXiv:2010.11929
12. **Oquab, M., Darcet, T., Moutakanni, T., et al.** (2023). *DINOv2: Learning Robust Visual Features without Supervision*. arXiv:2304.07193
13. **Liu, Z., Mao, H., Wu, C. Y., et al.** (2022). *A ConvNet for the 2020s (ConvNeXt)*. CVPR 2022. arXiv:2201.03545
14. **Hadsell, R., Chopra, S., & LeCun, Y.** (2006). *Dimensionality Reduction by Learning an Invariant Mapping*. CVPR 2006. *[Contrastive loss]*